

# **LAPORAN PRAKTIKUM STRUKTUR DATA**

## **PERTEMUAN 9**



**OLEH:**

**LEXI MULIA YUNASPI**

**2511531006**

**MATA KULIAH STRUKTUR DATA**

**DOSEN PENGAMPU : Dr, WAHYUDI ,M.T**

**ASISTEN PRAKTIKUM : M GHALID**

**FAKULTAS TEKNOLOGI INFORMASI**

**DEPARTEMEN INFORMATIKA**

**UNIVERSITAS ANDALAS**

**2026**

## **KATA PENGANTAR**

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk dokumentasi hasil praktikum mengenai implementasi algoritma Breadth First Search (BFS) dan Depth First Search (DFS) menggunakan bahasa pemrograman Java dengan antarmuka Graphical User Interface (GUI).

Praktikum ini bertujuan untuk memahami konsep graph serta penerapan algoritma BFS dan DFS dalam pencarian jalur pada suatu peta kampus. Selain itu, praktikum ini juga melatih kemampuan dalam merancang program berbasis objek serta membuat visualisasi graph menggunakan Java Swing.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa yang akan datang.

Padang, Juni 2026

Penulis

## DAFTAR ISI

|                                           |           |
|-------------------------------------------|-----------|
| <b>KATA PENGANTAR .....</b>               | <b>i</b>  |
| <b>DAFTAR ISI .....</b>                   | <b>ii</b> |
| <b>BAB I PENDAHULUAN .....</b>            | <b>1</b>  |
| 1.1 Latar Belakang .....                  | 1         |
| 1.2 Tujuan .....                          | 1         |
| 1.3 Manfaat .....                         | 1         |
| <b>BAB II PEMBAHASAN.....</b>             | <b>2</b>  |
| 2.1 Langkah Langkah praktikum .....       | 2         |
| 2.2 Langkah Langkah tugas praktikum ..... | 27        |
| <b>BAB III PENUTUP .....</b>              | <b>39</b> |
| 3.1 Kesimpulan .....                      | 39        |
| 3.2 Saran .....                           | 39        |
| <b>DAFTAR PUSTAKA .....</b>               | <b>40</b> |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Struktur data graph merupakan salah satu struktur data yang banyak digunakan untuk merepresentasikan hubungan antar objek. Graph dapat digunakan dalam berbagai bidang seperti jaringan komputer, sistem navigasi, media sosial, dan pencarian rute. Dalam pencarian jalur pada graph terdapat beberapa algoritma yang sering digunakan, di antaranya Breadth First Search (BFS) dan Depth First Search (DFS). BFS melakukan penelusuran secara melebar dari suatu node ke node lainnya, sedangkan DFS melakukan penelusuran secara mendalam hingga mencapai node tujuan. Pada praktikum ini dibuat aplikasi pencarian jalur pada peta kampus menggunakan Java GUI. Program mampu menampilkan visualisasi graph, memilih lokasi awal dan tujuan, serta mencari jalur menggunakan algoritma BFS dan DFS.

### **1.2 Tujuan**

Adapun tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami konsep graph dalam struktur data.
2. Memahami cara kerja algoritma BFS dan DFS.
3. Mengimplementasikan BFS dan DFS menggunakan Java.
4. Membuat visualisasi graph menggunakan Java Swing.

### **1.3 Manfaat**

Manfaat yang diperoleh dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Menambah pemahaman mengenai struktur data graph.
2. Memahami penerapan algoritma BFS dan DFS.
3. Melatih kemampuan pemrograman Java berbasis GUI.

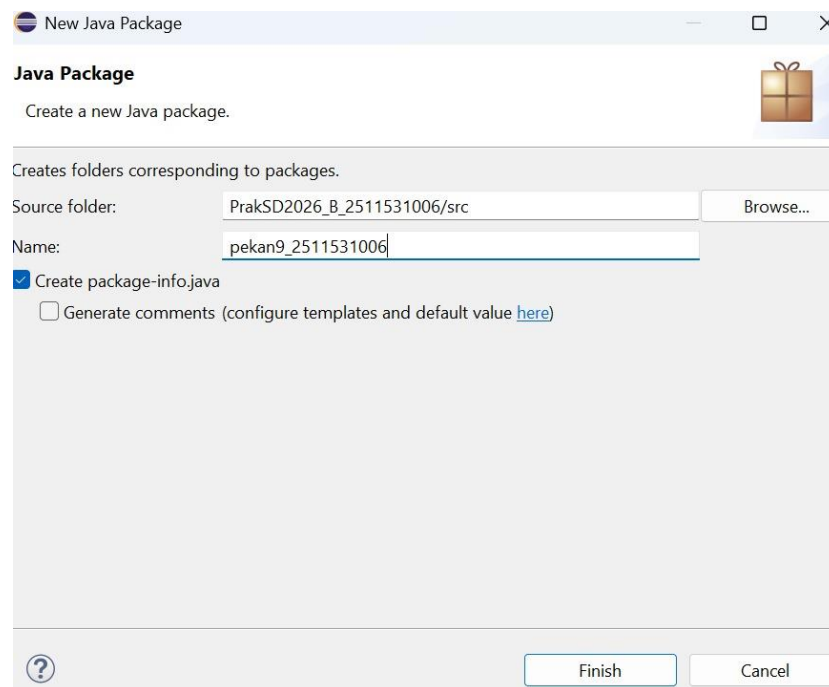
## BAB II

### PEMBAHASAN

#### 1.1 Langkah Langkah Praktikum

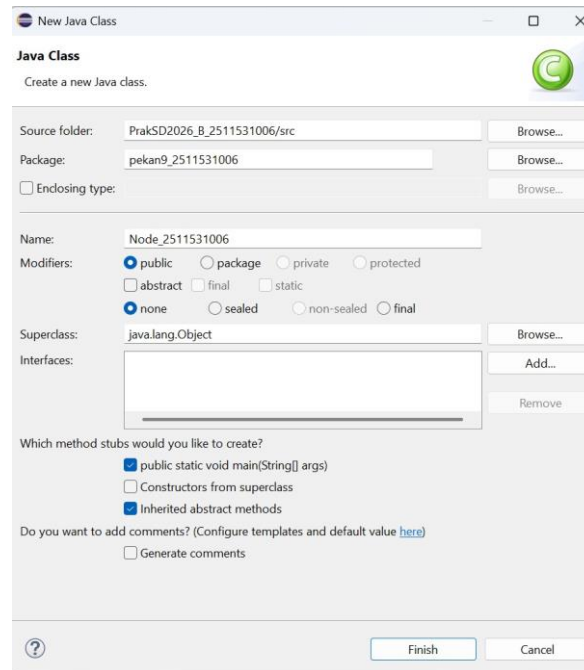
##### 1. Program 1

###### A. Package pekan9\_2511531006



Package pekan9\_2511531006 digunakan sebagai wadah untuk menyimpan seluruh class yang berkaitan dengan program Binary Tree. Penggunaan package membantu mengorganisasi file program agar lebih rapi dan mudah dikelola. Semua class yang berada dalam package yang sama dapat saling berinteraksi tanpa kendala akses tertentu. Package juga berfungsi untuk menghindari konflik nama class dengan program lain. Dengan adanya package, struktur proyek menjadi lebih teratur dan mudah dipahami.

###### B. Class Node\_1006



```
2  
3 public class Node_1006 {
```

Class Node\_1006 digunakan untuk merepresentasikan sebuah node pada struktur data Binary Tree. Setiap objek yang dibuat dari class ini akan menyimpan satu nilai data dan referensi ke child kiri serta child kanan. Class ini menjadi komponen utama dalam pembentukan pohon biner. Semua operasi yang berhubungan dengan node dilakukan melalui class ini. Dengan adanya class ini, data dapat disusun dalam bentuk struktur tree secara dinamis. Class ini juga menjadi dasar bagi proses traversal dan manipulasi tree.

### C. Deklarasi Variabel

```
4 int data_1006;  
5 Node_1006 left;  
6 Node_1006 right;  
7
```

Variabel data\_1006 digunakan untuk menyimpan nilai yang terdapat pada sebuah node. Variabel left digunakan untuk menyimpan alamat child kiri dari node saat ini. Variabel right digunakan untuk menyimpan alamat child kanan dari node saat ini. Kedua variabel child bertipe Node\_1006 karena menunjuk ke node lain dalam

Binary Tree. Ketiga variabel tersebut merupakan elemen utama yang membentuk hubungan antar node. Tanpa variabel ini, struktur Binary Tree tidak dapat dibangun.

#### D. Constructor Node\_1006()

```
8 public Node_1006(int data_1006) {  
9     this.data_1006 = data_1006;  
10    left = null;  
11    right = null;  
12 }
```

Constructor Node\_1006() digunakan untuk membuat objek node baru dan memberikan nilai awal pada node tersebut. Nilai yang diterima melalui parameter akan disimpan ke dalam variabel data\_1006. Sementara itu, variabel left dan right diinisialisasi dengan nilai null karena node belum memiliki child. Constructor akan dipanggil secara otomatis ketika objek dibuat menggunakan keyword new. Penggunaan constructor membuat proses pembuatan node menjadi lebih mudah dan efisien. Selain itu, constructor memastikan setiap node memiliki kondisi awal yang benar.

#### E. Method setLeft()

```
13 public void setLeft(Node_1006 node_1006) {  
14     if (left == null)  
15         left = node_1006;  
16 }
```

Method setLeft() digunakan untuk menambahkan child kiri pada sebuah node. Parameter yang diterima berupa objek Node\_1006 yang akan ditempatkan pada posisi kiri. Sebelum proses penyimpanan dilakukan, program memeriksa apakah child kiri masih kosong atau bernilai null. Jika posisi kiri masih kosong, maka node baru akan disimpan pada posisi tersebut. Method ini membantu proses pembentukan struktur Binary Tree secara bertahap. Dengan method ini, hubungan antara parent dan child kiri dapat dibentuk dengan mudah.

#### F. Method setRight()

```
17 public void setRight(Node_1006 node_1006) {  
18     if (right == null)  
19         right = node_1006;  
20 }
```

Method setRight() digunakan untuk menambahkan child kanan pada sebuah node. Method menerima parameter berupa objek Node\_1006 yang akan menjadi child kanan. Program terlebih dahulu memeriksa apakah posisi kanan masih kosong sebelum melakukan penyimpanan. Jika child kanan belum ada, maka node yang diberikan akan ditempatkan pada posisi tersebut. Method ini sangat penting dalam pembentukan cabang kanan pada Binary Tree. Dengan method ini, struktur pohon dapat dibangun secara lebih terkontrol.

#### G. Method getLeft(), getRight(), dan getData()

```
21 public Node_1006 getLeft() {  
22     return left;  
23 }  
24 public Node_1006 getRight() {  
25     return right;  
26 }  
27 public int getData() {  
28     return data_1006;  
29 }
```

Method getLeft(), getRight(), dan getData() merupakan method getter yang digunakan untuk mengambil nilai atribut dari objek. Method getLeft() mengembalikan child kiri, sedangkan getRight() mengembalikan child kanan. Method getData() digunakan untuk mengambil nilai data yang tersimpan pada node. Penggunaan getter mendukung konsep enkapsulasi dalam pemrograman berorientasi objek. Data dapat diakses secara aman tanpa harus mengakses variabel secara langsung. Selain itu, getter mempermudah proses pengambilan informasi dari suatu node.



#### H. Method setData()

```
30 public void setData(int data_1006) {  
31     this.data_1006 = data_1006;  
32 }  
33
```

Method setData() digunakan untuk mengubah nilai data yang tersimpan pada suatu node. Nilai baru diterima melalui parameter kemudian disimpan ke dalam variabel data\_1006. Method ini memungkinkan perubahan data tanpa perlu membuat objek node baru. Penggunaan setter merupakan salah satu penerapan konsep enkapsulasi dalam Java. Dengan setter, perubahan data dapat dikontrol melalui method tertentu. Hal ini membuat pengelolaan data menjadi lebih aman dan terstruktur.

#### I. Method printPreorder()

```
34 void printPreorder(Node_1006 node_1006) {  
35     if (node_1006 == null)  
36         return;  
37     System.out.print(node_1006.data_1006 + " ");  
38     printPreorder(node_1006.left);  
39     printPreorder(node_1006.right);  
40 }
```

Method printPreorder() digunakan untuk melakukan traversal Binary Tree dengan metode Preorder. Traversal dilakukan dengan urutan Root, Left, dan Right. Method bekerja secara rekursif dengan mengunjungi node akar terlebih dahulu sebelum berpindah ke child kiri dan child kanan. Jika node bernilai null, maka proses traversal pada cabang tersebut dihentikan. Hasil traversal akan ditampilkan ke layar menggunakan perintah System.out.print(). Metode ini sering digunakan untuk menyalin atau merepresentasikan struktur tree.

#### J. Method printPostorder()

```

41 void printPostorder(Node_1006 node_1006) {
42     if (node_1006 == null)
43         return;
44     printPostorder(node_1006.left);
45     printPostorder(node_1006.right);
46     System.out.print(node_1006.data_1006 + " ");
47 }

```

Method printPostorder() digunakan untuk melakukan traversal Binary Tree dengan metode Postorder. Traversal dilakukan dengan urutan Left, Right, dan Root. Seluruh node pada subtree kiri akan dikunjungi terlebih dahulu sebelum berpindah ke subtree kanan. Setelah kedua subtree selesai diproses, barulah node induk ditampilkan. Method ini menggunakan teknik rekursif untuk mengunjungi seluruh node dalam tree. Traversal Postorder sering digunakan pada proses penghapusan atau evaluasi struktur tree.

#### K. Method printInorder()

```

48 void printInorder(Node_1006 node_1006) {
49     if (node_1006 == null)
50         return;
51     printInorder(node_1006.left);
52     System.out.print(node_1006.data_1006 + " ");
53     printInorder(node_1006.right);
54 }

```

Method printInorder() digunakan untuk melakukan traversal Binary Tree dengan metode Inorder. Traversal dilakukan dengan urutan Left, Root, dan Right. Metode ini sangat sering digunakan pada Binary Search Tree karena menghasilkan data yang terurut dari nilai terkecil hingga terbesar. Method bekerja secara rekursif hingga seluruh node berhasil dikunjungi. Setiap node akan ditampilkan setelah subtree kiri selesai diproses. Hasil traversal ditampilkan menggunakan System.out.print().

#### L. Method print()

```

55 public String print() {
56     return this.print("", true, "");
57 }

```

Method `print()` digunakan sebagai method utama untuk menampilkan struktur Binary Tree dalam bentuk visual. Method ini akan memanggil method `print()` lainnya yang memiliki parameter tambahan. Pengguna tidak perlu mengisi parameter secara manual karena nilai awal sudah ditentukan dalam method ini. Method ini mempermudah proses pemanggilan fungsi cetak tree. Struktur tree dapat ditampilkan dengan satu kali pemanggilan method saja. Hasil tampilan akan menyerupai bentuk pohon pada console.

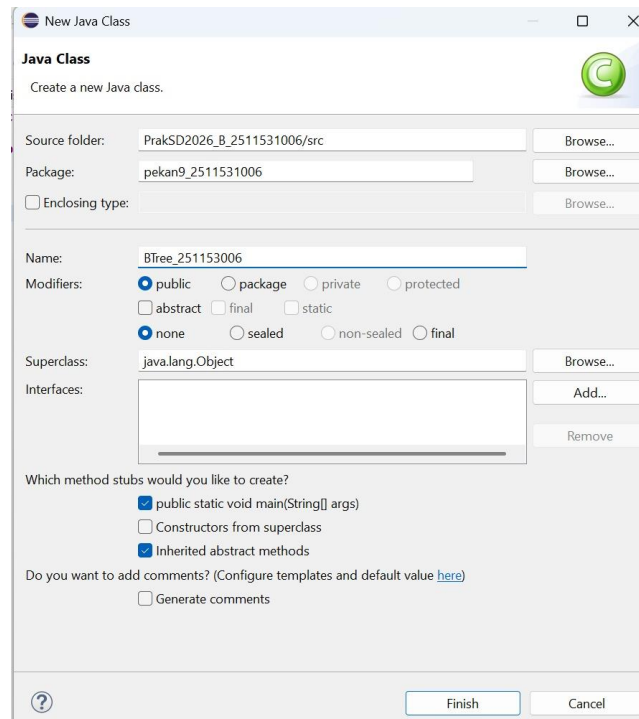
#### M. Method `print(String prefix, boolean isTail, String sb)`

```
58 public String print(String prefix, boolean isTail, String sb) {
59     if (right != null) {
60         right.print(prefix + (isTail ? "| " : " "), false, sb);
61     }
62     System.out.println(prefix + (isTail ? "\\-- " : "/-- ") + data_1006);
63     if (left != null) {
64         left.print(prefix + (isTail ? " " : "| "), true, sb);
65     }
66     return sb;
67 }
68 }
```

Method ini digunakan untuk menampilkan struktur Binary Tree secara visual menggunakan karakter cabang pada console. Parameter `prefix` digunakan untuk mengatur jarak atau indentasi tampilan setiap node. Parameter `isTail` digunakan untuk menentukan bentuk garis cabang yang akan dicetak. Method akan mencetak node kanan terlebih dahulu, kemudian node induk, lalu node kiri sehingga bentuk pohon lebih mudah dibaca. Proses pencetakan dilakukan secara rekursif hingga seluruh node berhasil ditampilkan. Method ini sangat membantu dalam proses visualisasi dan pengecekan struktur Binary Tree yang telah dibuat

## 2. Program 2

### A. Membuat Class `BTree_1006`



```
1 package pekan9_2511531006;
2
3 public class BTree_1006 {
4     private Node_1006 root_1006;
5     private Node_1006 currentNode_1006;
6     public BTree_1006() {
7         root_1006 = null;
8     }
```

Class **BTree\_1006** digunakan sebagai pengelola utama struktur data Binary Tree. Class ini berfungsi untuk menyimpan root tree, melakukan pencarian data, menghitung jumlah node, serta menampilkan isi tree dengan berbagai metode traversal. Selain itu, class ini menyediakan operasi untuk mengatur root dan node yang sedang aktif. Dengan adanya class ini, seluruh proses pengolahan Binary Tree dapat dilakukan secara terstruktur dan lebih mudah dikelola. Class BTree\_1006 bekerja sama dengan class Node\_1006 yang berfungsi sebagai pembentuk node pada tree.

#### B. Deklarasi Variabel Root dan Current Node

```
4     private Node_1006 root_1006;
5     private Node_1006 currentNode_1006;
```

Variabel **root\_1006** digunakan untuk menyimpan node pertama atau akar dari Binary Tree. Variabel **currentNode\_1006** digunakan untuk menyimpan node yang sedang diproses atau diakses pada saat tertentu. Kedua variabel dideklarasikan dengan tipe data **Node\_1006** karena akan menyimpan objek node. Penggunaan variabel ini memudahkan pengelolaan struktur tree selama proses berjalan. Variabel tersebut juga digunakan oleh berbagai operasi seperti pencarian, traversal, dan perhitungan jumlah node.

#### C. Membuat Constructor BTree\_1006

```
6 public BTree_1006() {  
7     root_1006 = null;  
8 }
```

Constructor digunakan untuk membuat objek Binary Tree yang baru. Saat objek dibuat, nilai root diinisialisasi menjadi null yang menandakan bahwa tree masih kosong dan belum memiliki node. Constructor ini akan dijalankan secara otomatis ketika objek BTree\_1006 dibuat. Dengan kondisi awal tersebut, program dapat mengetahui bahwa belum ada data yang dimasukkan ke dalam tree. Constructor menjadi langkah awal sebelum operasi lain dijalankan.

#### D. Membuat Proses Pencarian Data

```
9 public boolean search(int data_1006) {  
10     return search(root_1006, data_1006);  
11 }  
12 private boolean search(Node_1006 node_1006, int data_1006) {  
13     if (node_1006.getData() == data_1006)  
14         return true;  
15     if (node_1006.getLeft() != null)  
16         if (search(node_1006.getLeft(), data_1006))  
17             return true;  
18     if (node_1006.getRight() != null)  
19         if (search(node_1006.getRight(), data_1006))  
20             return true;  
21     return false;  
}
```

Pencarian data digunakan untuk mengetahui apakah suatu nilai terdapat di dalam Binary Tree atau tidak. Proses pencarian dilakukan secara rekursif dengan memeriksa node saat ini, kemudian melanjutkan ke subtree kiri dan kanan. Jika data ditemukan maka method akan mengembalikan nilai true. Sebaliknya, jika seluruh

node telah diperiksa dan data tidak ditemukan maka method akan mengembalikan nilai false. Proses ini memungkinkan pencarian data dilakukan secara menyeluruh pada tree.

#### E. Membuat Traversal Inorder

```
23 public void printInorder() {  
24     root_1006.printInorder(root_1006);  
25 }
```

Traversal Inorder digunakan untuk menampilkan data tree dengan urutan kiri, akar, kemudian kanan. Metode ini dilakukan secara rekursif hingga seluruh node selesai dikunjungi. Hasil traversal Inorder sering digunakan untuk menampilkan data dalam urutan tertentu. Pada class ini, proses traversal dilakukan dengan memanggil method `printInorder()` yang terdapat pada `Node_1006`. Seluruh data tree akan ditampilkan ke layar sesuai aturan Inorder.

#### F. Membuat Traversal Preorder

```
26 public void printPreOrder() {  
27     root_1006.printPreorder(root_1006);  
28 }
```

Traversal Preorder digunakan untuk menampilkan data dengan urutan akar, kiri, kemudian kanan. Proses ini dimulai dari node root terlebih dahulu sebelum mengunjungi child node. Traversal Preorder sering digunakan untuk menyalin atau merepresentasikan struktur tree. Pada program ini, proses dilakukan dengan memanggil method `printPreorder()` dari class `Node_1006`. Seluruh node akan ditampilkan sesuai urutan Preorder.

#### G. Membuat Traversal Postorder

```
29 public void printPostOrder() {  
30     root_1006.printPostorder(root_1006);  
31 }
```

Traversal Postorder digunakan untuk menampilkan data dengan urutan kiri, kanan, kemudian akar. Metode ini sering digunakan ketika proses membutuhkan

pengolahan child node terlebih dahulu sebelum parent node. Traversal dilakukan secara rekursif hingga seluruh node selesai dikunjungi. Pada program ini, proses dijalankan melalui method `printPostorder()` yang terdapat pada class `Node_1006`. Hasil traversal akan ditampilkan ke layar sesuai aturan Postorder.

#### H. Membuat Pengambilan Root Tree

```
32 public Node_1006 getRoot() {  
33     return root_1006;  
34 }
```

Pengambilan root digunakan untuk memperoleh node akar dari Binary Tree. Method ini mengembalikan objek `root_1006` yang menjadi titik awal seluruh struktur tree. Root sering digunakan oleh operasi lain seperti traversal dan pencarian data. Dengan adanya method ini, akses terhadap akar tree menjadi lebih mudah dan aman. Nilai yang dikembalikan bertipe `Node_1006`.

#### I. Membuat Pemeriksaan Tree Kosong

```
35 public boolean isEmpty() {  
36     return root_1006 == null;  
37 }
```

Pemeriksaan tree kosong digunakan untuk mengetahui apakah Binary Tree sudah memiliki node atau belum. Jika root bernilai `null` maka tree dianggap kosong. Sebaliknya, jika root memiliki nilai maka tree sudah berisi data. Pemeriksaan ini penting untuk menghindari kesalahan saat menjalankan operasi pada tree yang belum memiliki node. Hasil pemeriksaan dikembalikan dalam bentuk boolean.

#### J. Menghitung Jumlah Node

```

41 private int countNodes(Node_1006 node_1006) {
42     int count_1006 = 1;
43     if (node_1006 == null) {
44         return 0;
45     } else {
46         count_1006 += countNodes(node_1006.getLeft());
47         count_1006 += countNodes(node_1006.getRight());
48         return count_1006;
49     }
50 }

```

Perhitungan jumlah node digunakan untuk mengetahui banyaknya node yang terdapat di dalam Binary Tree. Proses dilakukan secara rekursif dengan menghitung node saat ini kemudian menjumlahkannya dengan jumlah node pada subtree kiri dan kanan. Jika node bernilai null maka nilai yang dikembalikan adalah 0. Hasil akhir menunjukkan total seluruh node yang ada dalam tree. Metode ini berguna untuk mengetahui ukuran tree yang sedang digunakan.

#### K. Menampilkan Struktur Binary Tree

```

51 public void print() {
52     root_1006.print();
53 }

```

Penampilan struktur Binary Tree digunakan untuk menampilkan bentuk tree secara visual melalui method print() yang terdapat pada class Node\_1006. Dengan method ini, hubungan antara parent dan child node dapat terlihat lebih jelas. Tampilan yang dihasilkan membantu pengguna memahami struktur tree yang telah dibuat. Method ini cukup memanggil fungsi print() pada root tree. Seluruh node akan ditampilkan sesuai bentuk percabangannya.

#### L. Membuat Getter dan Setter Current Node

```

54 public Node_1006 getCurrent() {
55     return currentNode_1006;
56 }
57 public void setCurrent(Node_1006 node_1006) {
58     this.currentNode_1006 = node_1006;
59 }

```

Getter dan setter digunakan untuk mengambil serta mengubah nilai currentNode\_1006. Variabel ini menyimpan node yang sedang diproses oleh program. Getter berfungsi mengembalikan objek currentNode\_1006, sedangkan setter digunakan untuk memberikan nilai baru pada variabel tersebut. Penggunaan



getter dan setter membuat pengelolaan data menjadi lebih terkontrol. Selain itu, kode program menjadi lebih terstruktur dan mudah dipelihara.

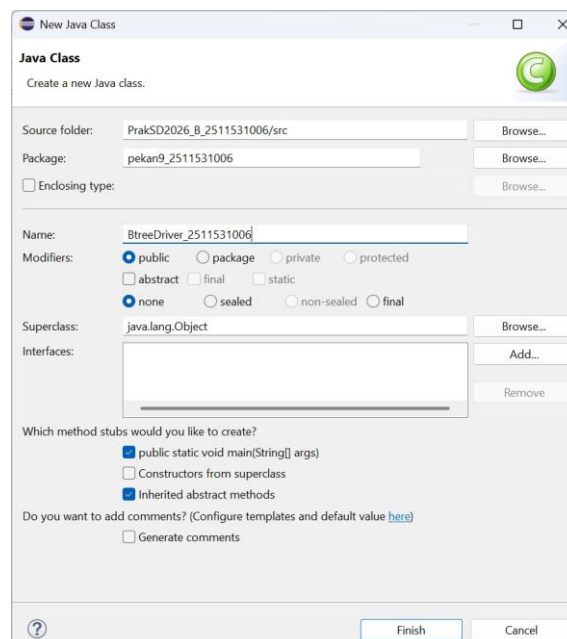
#### M. Membuat Setter Root Tree

```
60 public void setRoot(Node_1006 root_1006) {  
61     this.root_1006 = root_1006;  
62 }  
63 }
```

Setter root digunakan untuk mengubah atau menetapkan node akar pada Binary Tree. Method ini menerima parameter berupa objek Node\_1006 yang akan dijadikan root baru. Penggunaan setter memudahkan proses pembentukan dan pengaturan struktur tree. Setelah root diubah, seluruh operasi tree akan menggunakan node tersebut sebagai titik awal. Method ini mendukung fleksibilitas dalam pengelolaan Binary Tree.

### 3. Program 3

#### A. Membuat Class BtreeDriver\_1006



Class BtreeDriver\_1006 digunakan sebagai program utama untuk menjalankan seluruh operasi pada Binary Tree. Class ini berisi method main() yang menjadi titik awal eksekusi program. Melalui class ini dilakukan pembuatan objek tree, penambahan node, pengaturan hubungan antar node, serta pengujian traversal tree. Selain itu, class ini digunakan untuk menampilkan jumlah simpul dan bentuk pohon yang telah dibuat. Dengan adanya class driver, seluruh fungsi Binary Tree dapat dijalankan dan diuji secara langsung.

## B. Membuat Objek Binary Tree

```
6 // Membuat Pohon
7 BTree_1006 tree_1006 = new BTree_1006();
```

Objek Binary Tree dibuat menggunakan class BTree\_1006. Objek ini berfungsi sebagai wadah utama untuk menyimpan seluruh node yang akan membentuk pohon biner. Setelah objek dibuat, program dapat melakukan berbagai operasi seperti menambahkan root, menghitung jumlah node, dan menampilkan traversal. Objek tree menjadi pusat pengelolaan seluruh data pada Binary Tree. Semua proses selanjutnya dilakukan melalui objek tersebut.

## C. Menampilkan Jumlah Simpul Awal Pohon

```
8 System.out.print("Jumlah Simpul awal pohon: ");
9 System.out.println(tree_1006.countNodes());
```

Jumlah simpul awal ditampilkan untuk mengetahui kondisi Binary Tree sebelum memiliki node. Karena root belum dibuat, jumlah node yang tersimpan masih bernilai nol. Langkah ini digunakan untuk memastikan bahwa tree dalam keadaan kosong saat pertama kali dijalankan. Perhitungan dilakukan menggunakan method countNodes(). Hasilnya kemudian ditampilkan ke layar.

#### D. Membuat Node Root

```
10 // menambahkan simpul data 1
11 Node_1006 root_1006 = new Node_1006(1);
```

Node root dibuat sebagai simpul pertama pada Binary Tree. Root berfungsi sebagai akar yang menjadi titik awal seluruh percabangan tree. Pada program ini root diberikan nilai data 1. Node dibuat menggunakan constructor dari class Node\_1006. Setelah dibuat, node tersebut siap digunakan sebagai akar pohon.

#### E. Menetapkan Root ke Dalam Tree

```
12 // menjadikan simpul 1 sebagai root
13 tree_1006.setRoot(root_1006);
```

Node root yang telah dibuat kemudian ditetapkan sebagai akar Binary Tree menggunakan method setRoot(). Setelah root ditetapkan, tree tidak lagi berada dalam keadaan kosong. Seluruh node lain nantinya akan terhubung melalui root tersebut. Root menjadi titik awal berbagai operasi seperti traversal dan pencarian data. Dengan langkah ini struktur tree mulai terbentuk.

#### F. Menampilkan Jumlah Simpul Setelah Root Ditambahkan

```
14 System.out.println("Jumlah simpul jika hanya ada root");
15 System.out.println(tree_1006.countNodes());
```

Jumlah simpul ditampilkan kembali untuk memastikan bahwa root berhasil dimasukkan ke dalam tree. Karena hanya terdapat satu node yaitu root, maka hasil yang ditampilkan adalah satu. Langkah ini digunakan sebagai proses pengecekan keberhasilan penambahan root. Method countNodes() menghitung jumlah seluruh node yang ada dalam tree. Hasilnya kemudian ditampilkan pada console.

#### G. Membuat Node-Node Baru

```
16 Node_1006 node2_1006 = new Node_1006(2);
17 Node_1006 node3_1006 = new Node_1006(3);
18 Node_1006 node4_1006 = new Node_1006(4);
19 Node_1006 node5_1006 = new Node_1006(5);
20 Node_1006 node6_1006 = new Node_1006(6);
21 Node_1006 node7_1006 = new Node_1006(7);
22 Node_1006 node8_1006 = new Node_1006(8);
23 Node_1006 node9_1006 = new Node_1006(9);
```

Beberapa node baru dibuat untuk membentuk struktur Binary Tree yang lebih lengkap. Setiap node memiliki nilai data yang berbeda mulai dari 2 hingga 9. Node-node tersebut nantinya akan dihubungkan sebagai child kiri atau child kanan dari node lainnya. Pembuatan node dilakukan menggunakan constructor dari class Node\_1006. Dengan adanya node-node ini, tree dapat membentuk percabangan yang lebih kompleks.

#### H. Menghubungkan Node Menjadi Binary Tree

```
24 root_1006.setLeft(node2_1006);
25 node2_1006.setLeft(node4_1006);
26 node2_1006.setRight(node5_1006);
27 node4_1006.setRight(node8_1006);
28 root_1006.setRight(node3_1006);
29 node3_1006.setLeft(node6_1006);
30 node3_1006.setRight(node7_1006);
31 node6_1006.setLeft(node9_1006);
```

Setelah seluruh node dibuat, langkah berikutnya adalah menghubungkan node-node tersebut menjadi sebuah Binary Tree. Hubungan dilakukan dengan menentukan child kiri dan child kanan menggunakan method setLeft() dan setRight(). Proses ini membentuk percabangan yang menunjukkan hubungan antar node. Setiap node dapat memiliki maksimal dua child. Setelah proses selesai, struktur Binary Tree terbentuk dengan lengkap.

#### I. Menentukan Current Node

```
33 // Set root
34 tree_1006.setCurrent(tree_1006.getRoot());
```

Current node digunakan untuk menyimpan node yang sedang aktif atau sedang diproses oleh program. Pada bagian ini current node diisi dengan root tree sehingga node pertama yang aktif adalah akar pohon. Variabel current node memudahkan program dalam mengakses node tertentu. Nilainya dapat diambil kembali menggunakan method `getCurrent()`. Penggunaan current node membuat pengelolaan tree menjadi lebih teratur.

#### J. Menampilkan Current Node

```
35         System.out.println("Menampilkan simpul terakhir: ");
36         System.out.println(tree_1006.getCurrent().getData());
```

Data pada current node ditampilkan untuk memastikan bahwa node aktif telah berhasil ditetapkan. Karena current node berisi root, maka nilai yang ditampilkan adalah data root yaitu 1. Informasi ini digunakan untuk memverifikasi proses sebelumnya. Data diperoleh menggunakan method `getData()`. Hasilnya kemudian dicetak ke layar.

#### K. Menampilkan Jumlah Seluruh Simpul

```
37         System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
38         System.out.println(tree_1006.countNodes());
```

Jumlah simpul ditampilkan kembali setelah seluruh node selesai ditambahkan ke dalam tree. Perhitungan dilakukan secara rekursif mulai dari root hingga seluruh child node. Hasil yang diperoleh menunjukkan total node yang terdapat pada Binary Tree. Informasi ini berguna untuk mengetahui ukuran tree yang telah dibuat. Nilai kemudian ditampilkan pada console.

#### L. Menampilkan Traversal Inorder

```
39         System.out.println("InOrder: ");
40         tree_1006.printInorder();
```

Traversal Inorder digunakan untuk mengunjungi node dengan urutan kiri, akar, lalu kanan. Proses traversal dilakukan secara rekursif hingga seluruh node selesai dikunjungi. Traversal ini sering digunakan untuk menampilkan data secara terurut pada Binary Search Tree. Hasil traversal akan dicetak ke layar sesuai aturan Inorder. Seluruh node ditampilkan satu per satu sesuai urutannya.

#### M. Menampilkan Traversal Preorder

```
41      System.out.println("\nPreorder: ");  
42      tree_1006.printPreOrder();
```

Traversal Preorder dilakukan dengan urutan akar, kiri, lalu kanan. Root akan dikunjungi terlebih dahulu sebelum berpindah ke child node lainnya. Traversal ini sering digunakan untuk menggambarkan struktur tree dari atas ke bawah. Proses dilakukan secara rekursif hingga seluruh node selesai dikunjungi. Hasil traversal ditampilkan pada console.

#### N. Menampilkan Traversal Postorder

```
43      System.out.println("\nPostorder : ");  
44      tree_1006.printPostOrder();
```

Traversal Postorder dilakukan dengan urutan kiri, kanan, lalu akar. Child node akan diproses terlebih dahulu sebelum parent node ditampilkan. Traversal ini sering digunakan untuk proses penghapusan tree atau evaluasi struktur tertentu. Seluruh node dikunjungi secara rekursif hingga selesai. Hasil traversal kemudian ditampilkan ke layar.

#### O. Menampilkan Bentuk Binary Tree

```
45      System.out.println("\nMenampilkan simpul dalam bentuk pohon");
```

Struktur Binary Tree dapat ditampilkan dalam bentuk percabangan sehingga hubungan antar node terlihat dengan jelas. Tampilan ini membantu pengguna memahami posisi root, child kiri, dan child kanan. Proses dilakukan menggunakan

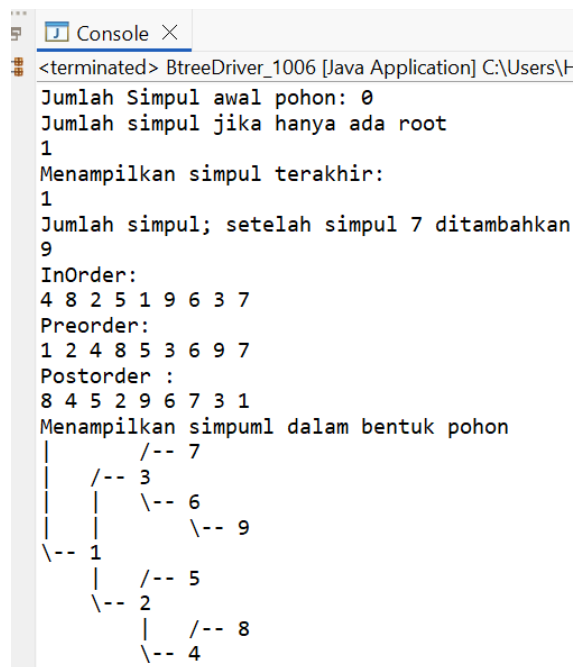
method print() yang terdapat pada class BTree\_1006. Seluruh node akan dicetak sesuai bentuk pohonnya. Hasil akhirnya ditampilkan pada console.

#### P. Menampilkan Hasil Akhir Struktur Pohon

```
46         tree_1006.print();
47     }
48 }
```

Tahap terakhir adalah menampilkan keseluruhan struktur Binary Tree dalam bentuk visual percabangan. Tampilan ini menunjukkan hubungan antara root dengan seluruh child yang dimilikinya. Dengan melihat bentuk pohon, pengguna dapat memahami susunan node yang telah dibuat. Proses dilakukan melalui method print() pada objek tree. Hasil akhir ditampilkan dalam bentuk struktur Binary Tree pada console.

#### Q. Hasil Output Program



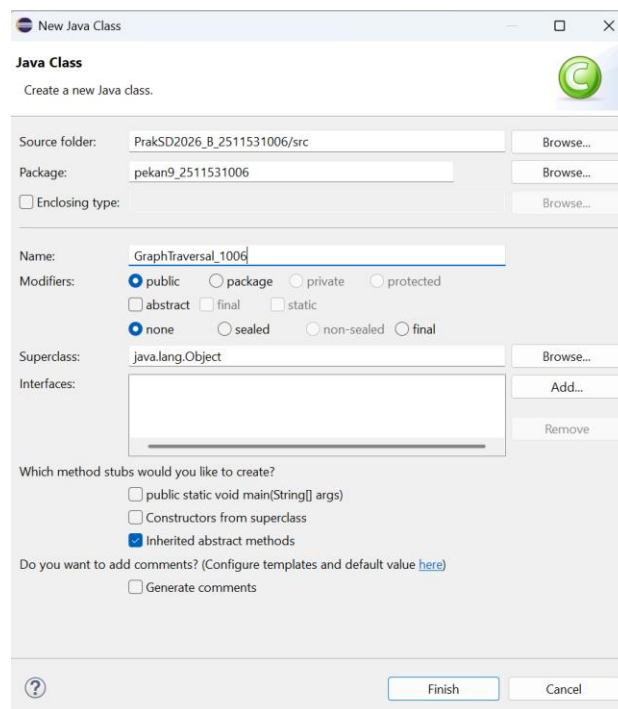
```
<terminated> BtreeDriver_1006 [Java Application] C:\Users\H
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
Menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder :
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|
| /-- 7
| /-- 3
| | \-- 6
| | \-- 9
|-- 1
|   /-- 5
|   \-- 2
|       /-- 8
|       \-- 4
```

Program berhasil membuat struktur Binary Tree yang terdiri dari sembilan simpul dengan data 1 sampai 9. Setelah seluruh simpul dihubungkan, program menampilkan jumlah simpul yang ada di dalam pohon, yaitu sebanyak 9 simpul. Selanjutnya dilakukan traversal InOrder, PreOrder, dan PostOrder untuk

menampilkan urutan kunjungan setiap simpul berdasarkan metode yang berbeda. Program juga menampilkan bentuk pohon secara visual menggunakan karakter ASCII sehingga hubungan antara root, child kiri, dan child kanan dapat terlihat dengan jelas. Hasil output menunjukkan bahwa seluruh operasi pada Binary Tree berjalan sesuai dengan rancangan program yang dibuat.

#### 4. Program 4

##### A. Membuat Class GraphTraversal\_1006



```
4 public class GraphTraversal_1006 {  
5
```

Class GraphTraversal\_1006 digunakan untuk merepresentasikan struktur data graf dan mengimplementasikan algoritma penelusuran DFS (*Depth First Search*) serta BFS (*Breadth First Search*). Pada class ini digunakan struktur data HashMap untuk menyimpan hubungan antar simpul dalam bentuk *adjacency list*. Selain itu, class ini juga menyediakan fungsi untuk menambahkan sisi (*edge*), menampilkan graf, dan melakukan proses penelusuran. Dengan adanya class ini, proses pengelolaan dan pencarian jalur pada graf dapat dilakukan dengan lebih mudah dan terstruktur.



## B. Membuat Variabel Graph

```
5  
6     private Map<String, List<String>> graph_1006 = new HashMap<>();  
7
```

Variabel `graph_1006` digunakan untuk menyimpan data graf dalam bentuk *adjacency list*. Struktur `Map<String, List<String>>` memungkinkan setiap simpul memiliki daftar tetangga yang terhubung dengannya. Penggunaan `HashMap` membuat proses pencarian dan penyimpanan data menjadi lebih cepat dan efisien. Setiap simpul direpresentasikan dengan tipe data `String`, sedangkan hubungan antar simpul disimpan dalam `ArrayList`. Dengan struktur ini, graf dapat dibangun secara dinamis sesuai kebutuhan program.

## C. Import Library Java Util

```
2  
3     import java.util.*;
```

Library `java.util` digunakan untuk menyediakan berbagai struktur data yang dibutuhkan dalam program. Struktur data seperti `Map`, `HashMap`, `List`, `ArrayList`, `Queue`, `LinkedList`, dan `HashSet` digunakan untuk membangun graf serta menjalankan algoritma DFS dan BFS. Dengan memanfaatkan library ini, penulisan program menjadi lebih sederhana dan efisien. Selain itu, library ini sudah tersedia secara bawaan pada Java sehingga tidak memerlukan instalasi tambahan.

## D. Membuat Method addEdge

```
90     public void addEdge(String node1_1006, String node2_1006) {
```

Method `addEdge()` digunakan untuk menambahkan hubungan atau sisi (*edge*) antara dua simpul pada graf. Karena graf yang digunakan bersifat tidak berarah (*undirected graph*), maka hubungan ditambahkan ke kedua simpul. Method ini juga memastikan bahwa simpul sudah tersedia pada `HashMap` sebelum hubungan ditambahkan. Dengan cara ini, graf dapat dibangun secara bertahap sesuai data yang diberikan. Setiap pemanggilan method akan menambahkan satu hubungan baru ke dalam graf.

#### E. Menambahkan Simpul ke Graph

```
10         graph_1006.putIfAbsent(node1_1006, new ArrayList<>());  
11         graph_1006.putIfAbsent(node2_1006, new ArrayList<>());
```

Instruksi ini digunakan untuk membuat daftar tetangga baru apabila simpul belum ada di dalam graf. Method `putIfAbsent()` akan memeriksa keberadaan simpul sebelum membuat `ArrayList` baru. Dengan demikian, tidak akan terjadi duplikasi simpul pada struktur data graf. Proses ini sangat penting agar hubungan antar simpul dapat disimpan dengan benar. Setelah simpul tersedia, barulah hubungan antar simpul dapat ditambahkan.

#### F. Menambahkan Hubungan Antar Simpul

```
12         graph_1006.get(node1_1006).add(node2_1006);  
13         graph_1006.get(node2_1006).add(node1_1006);
```

Bagian ini digunakan untuk menyimpan hubungan antar simpul pada graf. Karena graf bersifat tidak berarah, maka node pertama ditambahkan ke daftar tetangga node kedua dan sebaliknya. Dengan cara ini, setiap simpul dapat mengetahui simpul lain yang terhubung dengannya. Data hubungan tersebut nantinya digunakan dalam proses DFS dan BFS. Hubungan yang tersimpan membentuk struktur graf yang akan ditelusuri.

#### G. Membuat Method `printGraph`

```
16     public void printGraph() {
```

Method `printGraph()` digunakan untuk menampilkan isi graf dalam bentuk *adjacency list*. Setiap simpul akan ditampilkan beserta daftar tetangga yang terhubung dengannya. Tampilan ini memudahkan pengguna untuk melihat struktur graf yang telah dibuat. Selain itu, hasil tampilan dapat digunakan untuk memastikan bahwa hubungan antar simpul sudah tersimpan dengan benar. Method ini hanya berfungsi untuk menampilkan data tanpa mengubah isi graf.

#### H. Menampilkan Judul Graph

```
17 System.out.println("Graf Awal (Adjacency List):");
```

Perintah ini digunakan untuk menampilkan teks judul sebelum data graf dicetak ke layar. Tujuannya adalah agar pengguna dapat memahami bahwa data yang akan ditampilkan merupakan representasi graf dalam bentuk *adjacency list*. Tampilan yang jelas membuat hasil program lebih mudah dibaca dan dipahami. Judul ini muncul sebelum daftar hubungan antar simpul ditampilkan.

#### I. Menampilkan Isi Graph

```
18 for (String node_1006 : graph_1006.keySet()) {
```

Perulangan digunakan untuk mengakses seluruh simpul yang terdapat pada graf. Setiap simpul kemudian ditampilkan bersama daftar tetangganya. Dengan cara ini, seluruh hubungan yang tersimpan dalam graf dapat dilihat secara lengkap. Hasil yang ditampilkan menunjukkan struktur hubungan antar simpul yang akan digunakan pada proses DFS dan BFS.

#### J. Membuat Method DFS

```
27 public void dfs(String start_1006) {
```

Method `dfs()` digunakan untuk melakukan penelusuran graf menggunakan algoritma *Depth First Search*. Algoritma ini akan menelusuri simpul sedalam mungkin sebelum kembali ke simpul sebelumnya. Untuk menghindari kunjungan berulang, digunakan `HashSet` sebagai penyimpan simpul yang telah dikunjungi. Hasil penelusuran kemudian ditampilkan ke layar sesuai urutan kunjungan. DFS sangat cocok digunakan untuk pencarian jalur atau eksplorasi graf.

#### K. Membuat HashSet pada DFS

```
28 Set<String> visited_1006 = new HashSet<>();
```

Variabel `visited_1006` digunakan untuk menyimpan data simpul yang telah dikunjungi selama proses DFS berlangsung. Dengan adanya variabel ini, program dapat menghindari kunjungan berulang ke simpul yang sama. Hal tersebut penting untuk mencegah terjadinya perulangan tak berujung pada graf yang memiliki siklus. Data yang tersimpan dalam `HashSet` juga dapat diakses dengan cepat.

#### L. Membuat Method `dfsHelper`

```
33 private void dfsHelper(String current_1006, Set<String> visited_1006) {
```

Method `dfsHelper()` merupakan method rekursif yang digunakan untuk menjalankan proses DFS secara bertahap. Method ini akan memeriksa apakah suatu simpul sudah dikunjungi atau belum. Jika belum, simpul akan ditandai sebagai telah dikunjungi dan ditampilkan ke layar. Selanjutnya method akan memanggil dirinya sendiri untuk mengunjungi seluruh tetangga yang terhubung. Proses ini berlangsung hingga seluruh simpul yang dapat dijangkau berhasil dikunjungi.

#### M. Membuat Method BFS

```
43 public void bfs(String start_1006) {
```

Method `bfs()` digunakan untuk melakukan penelusuran graf menggunakan algoritma *Breadth First Search*. Berbeda dengan DFS, BFS menelusuri graf berdasarkan tingkat kedalaman atau level. Simpul yang paling dekat dengan titik awal akan dikunjungi terlebih dahulu. Algoritma ini menggunakan struktur data `Queue` untuk mengatur urutan kunjungan simpul. Hasil penelusuran akan ditampilkan sesuai urutan BFS.

#### N. Membuat Queue pada BFS

```
45 Queue<String> queue_1006 = new LinkedList<>();
```

`Queue` digunakan untuk menyimpan simpul yang akan dikunjungi selama proses BFS berlangsung. Struktur data ini bekerja dengan prinsip FIFO (*First In First Out*), sehingga simpul yang masuk lebih dahulu akan diproses terlebih dahulu.

Penggunaan queue memungkinkan BFS melakukan penelusuran per level secara teratur. Dengan demikian, seluruh simpul dapat dikunjungi sesuai aturan algoritma BFS.

O. Menambahkan Simpul Awal ke Queue

```
46         queue_1006.add(start_1006);  
47         visited_1006.add(start_1006);
```

Bagian ini digunakan untuk memasukkan simpul awal ke dalam queue sekaligus menandainya sebagai telah dikunjungi. Langkah ini menjadi titik awal proses BFS. Setelah simpul awal diproses, seluruh tetangganya akan ditambahkan ke queue untuk dikunjungi berikutnya. Dengan demikian, penelusuran dapat berjalan secara berurutan berdasarkan level.

P. Membuat Objek Graph pada Main

```
63         GraphTraversal_1006 graph_1006 = new GraphTraversal_1006();  
64
```

Objek graph\_1006 dibuat dari class GraphTraversal\_1006 untuk menjalankan seluruh fungsi yang telah dibuat sebelumnya. Objek ini menjadi representasi graf yang akan digunakan dalam program. Setelah objek dibuat, pengguna dapat menambahkan simpul, hubungan antar simpul, serta menjalankan DFS dan BFS. Tanpa objek ini, method yang ada pada class tidak dapat digunakan.

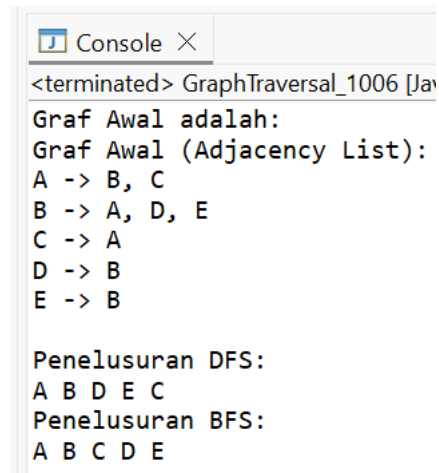
Q. Menambahkan Data Graph

```
65         // Contoh graf : A-B, A-C, B-D, B-E  
66         graph_1006.addEdge("A", "B");  
67         graph_1006.addEdge("A", "C");  
68         graph_1006.addEdge("B", "D");  
69         graph_1006.addEdge("B", "E");
```

Bagian ini digunakan untuk membangun struktur graf dengan menambahkan beberapa hubungan antar simpul. Hubungan yang dibuat adalah A-B, A-C, B-D, dan B-E. Dari hubungan tersebut terbentuk graf sederhana yang terdiri dari lima simpul. Data inilah yang nantinya digunakan sebagai contoh untuk proses DFS dan

BFS. Semakin banyak hubungan yang ditambahkan, semakin kompleks struktur graf yang terbentuk.

#### R. Hasil Program



```
<terminated> GraphTraversal_1006 [Ja
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

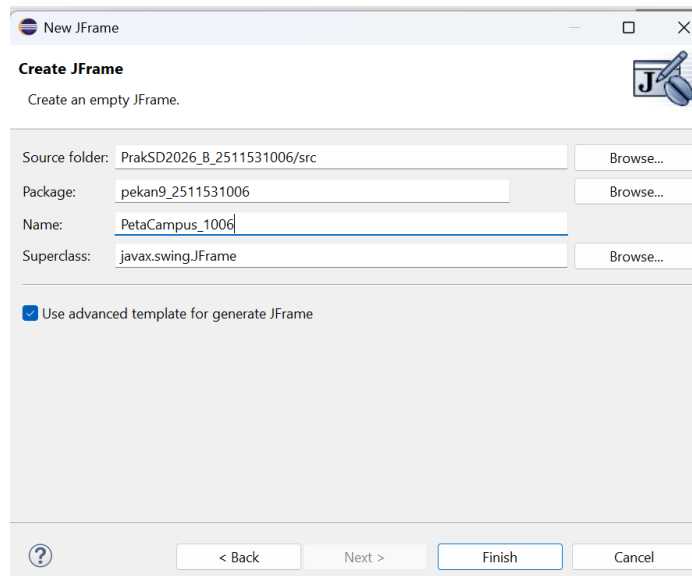
Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

Program berhasil membentuk graf dengan lima simpul yaitu A, B, C, D, dan E. Struktur graf ditampilkan dalam bentuk *adjacency list* sehingga hubungan antar simpul dapat dilihat dengan jelas. Setelah itu dilakukan penelusuran menggunakan algoritma DFS yang menghasilkan urutan kunjungan A, B, D, E, dan C. Selanjutnya algoritma BFS menghasilkan urutan kunjungan A, B, C, D, dan E sesuai level kedekatan dari simpul awal. Hasil tersebut menunjukkan bahwa implementasi DFS dan BFS telah berjalan dengan benar sesuai teori traversal graf.

## 1.2 Langkah langkah Tugas Praktikum

### 1. Program 1

A. Membuat Class PetaCampus\_1006



```
10  
11 public class PetaCampus_1006 extends JFrame {  
12
```

Class `PetaCampus_1006` merupakan class utama yang digunakan untuk menjalankan seluruh proses pencarian jalur pada peta kampus menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS). Class ini mewarisi `JFrame` sehingga dapat menampilkan antarmuka grafis berbasis Java Swing. Di dalam class terdapat struktur graph, komponen GUI, algoritma pencarian, serta visualisasi jalur yang ditemukan. Seluruh proses pengolahan data lokasi kampus dilakukan pada class ini. Class ini juga menjadi pusat pengendali semua aktivitas yang terjadi pada aplikasi.

## B. Import Library

```
3 import javax.swing.*;  
4 import javax.swing.border.*;  
5 import java.awt.*;  
6 import java.awt.event.*;  
7 import java.awt.geom.*;  
8 import java.util.*;  
9 import java.util.List;
```

Library yang digunakan berfungsi untuk mendukung pembuatan antarmuka grafis, pengelolaan event, pengaturan warna, font, koordinat, serta struktur data graph.

Package Swing menyediakan komponen GUI seperti JFrame, JLabel, JButton, dan JComboBox. Package AWT digunakan untuk mengatur tampilan visual aplikasi. Sementara Collection Framework dari Java digunakan untuk menyimpan node, edge, jalur pencarian, dan data graph. Seluruh library tersebut diperlukan agar aplikasi dapat berjalan dengan baik.

### C. Deklarasi Variabel Graph

```
private Map<String, List<String>> graph_1006 = new LinkedHashMap<>();
private Map<String, Point> posisiNode_1006 = new LinkedHashMap<>();
private List<String> jalurDitemukan_1006 = new ArrayList<>();
private List<String> nodeDikunjungi_1006 = new ArrayList<>();
private Set<String> nodeVisited_1006 = new HashSet<>();
private Set<String> nodeOnPath_1006 = new HashSet<>();
private String startNode_1006 = "";
private String goalNode_1006 = "";
```

Variabel graph digunakan untuk menyimpan seluruh hubungan antar lokasi kampus dalam bentuk adjacency list. Variabel posisi node digunakan untuk menentukan letak node pada panel visualisasi graph. Selain itu terdapat variabel untuk menyimpan node yang telah dikunjungi, jalur yang ditemukan, serta node yang termasuk dalam jalur solusi. Variabel lokasi awal dan tujuan juga disimpan untuk mendukung proses pencarian. Semua variabel ini menjadi dasar utama dalam proses BFS dan DFS.

### D. Deklarasi Komponen GUI

```
private JComboBox<String> comboAwal_1006;
private JComboBox<String> comboTujuan_1006;
private PanelGraph_1006 panelGraph_1006;
private JLabel labelJalur_1006;
private JLabel labelNodeDikunjungi_1006;
private JLabel labelJumlahNode_1006;
private JLabel labelHasil_1006;
```

Komponen GUI digunakan sebagai media interaksi antara pengguna dan aplikasi. ComboBox digunakan untuk memilih lokasi awal dan lokasi tujuan pencarian jalur. Label digunakan untuk menampilkan hasil pencarian, jalur yang ditemukan, jumlah node yang dikunjungi, dan informasi lainnya. Panel graph digunakan untuk menampilkan visualisasi graph secara langsung. Seluruh komponen ini akan ditampilkan pada jendela utama program.



#### E. Konstruktor Class

```
public PetaCampus_1006() {  
    initGraph_1006();  
    initGUI_1006();  
}
```

Konstruktor digunakan untuk menjalankan proses awal ketika objek PetaCampus\_1006 dibuat. Saat konstruktor dipanggil, program akan membangun graph lokasi kampus dan membuat seluruh tampilan GUI. Dengan cara ini pengguna dapat langsung menggunakan aplikasi setelah program dijalankan. Konstruktor berfungsi sebagai titik awal inisialisasi seluruh komponen aplikasi. Semua data dan tampilan dipersiapkan melalui konstruktor ini.

#### F. Membuat Data Graph Kampus

```
private void initGraph_1006() {
```

Method ini digunakan untuk membangun graph yang berisi lokasi-lokasi kampus. Node yang dibuat meliputi Gerbang Utama, Rektorat, Perpustakaan, FTI, FKIP, Laboratorium, Kantin, Masjid, dan lokasi lainnya. Selain membuat node, method ini juga menentukan hubungan antar lokasi serta posisi masing-masing node pada panel visualisasi. Data graph yang dibuat nantinya akan digunakan oleh algoritma BFS dan DFS. Struktur graph ini menjadi representasi peta kampus pada aplikasi.

#### G. Menambahkan Edge Antar Node

```
private void addEdge_1006(String a, String b) {  
    graph_1006.get(a).add(b);  
    graph_1006.get(b).add(a);  
}
```

Method ini digunakan untuk menghubungkan dua lokasi dalam graph. Karena graph bersifat tidak berarah, maka setiap node akan saling terhubung dua arah. Saat sebuah edge ditambahkan, kedua node akan masuk ke daftar tetangga masing-

masing. Dengan demikian BFS dan DFS dapat berpindah dari satu lokasi ke lokasi lainnya. Method ini dipanggil berkali-kali saat membangun graph kampus.

#### H. Membuat Tampilan GUI

```
private void initGUI_1006() {
```

Method ini berfungsi untuk membangun seluruh tampilan aplikasi yang meliputi header, panel kontrol, tombol BFS, tombol DFS, tombol Reset, panel graph, dan panel hasil pencarian. Ukuran jendela, warna tema, font, serta tata letak komponen juga diatur pada method ini. Semua komponen antarmuka ditempatkan pada posisi yang sesuai agar mudah digunakan pengguna. Tampilan GUI dibuat menggunakan Java Swing dengan konsep event-driven programming. Method ini menjadi bagian terbesar dalam pembuatan antarmuka aplikasi.

#### I. Membuat Tombol

```
private JButton buatTombol_1006(String teks, Color bg, Color fg) {
```

Method ini digunakan untuk membuat tombol dengan desain yang seragam. Setiap tombol diberikan warna, font, border, dan efek kursor yang sama sehingga tampilan aplikasi menjadi lebih rapi. Tombol yang dibuat digunakan untuk menjalankan BFS, DFS, dan Reset. Penggunaan method ini mengurangi pengulangan kode dalam pembuatan tombol. Dengan demikian program menjadi lebih efisien dan mudah dipelihara.

#### J. Membuat Label Hasil

```
private JLabel buatLabelHasil_1006(String teks, Font f, Color fg, Color bg) {
```

Method ini digunakan untuk membuat label yang akan menampilkan informasi hasil pencarian. Label dibuat dengan warna dan font tertentu agar mudah dibaca pengguna. Informasi yang ditampilkan meliputi hasil pencarian, jalur yang ditemukan, node yang dikunjungi, dan jumlah node yang diperiksa. Penggunaan

method ini membuat pembuatan label menjadi lebih sederhana dan konsisten. Semua label hasil dibuat melalui method ini.

#### K. Implementasi Algoritma BFS

```
private void jalankanBFS_1006() {
```

Method `jalankanBFS_1006()` digunakan untuk melakukan pencarian jalur menggunakan algoritma Breadth First Search. Algoritma BFS bekerja dengan memanfaatkan Queue sehingga pencarian dilakukan berdasarkan tingkat kedalaman yang sama terlebih dahulu. Selama proses berjalan, setiap node yang dikunjungi akan dicatat dan parent node akan disimpan. Setelah node tujuan ditemukan, jalur akan direkonstruksi berdasarkan data parent tersebut. Hasil pencarian kemudian ditampilkan pada GUI.

#### L. Implementasi Algoritma DFS

```
private void jalankanDFS_1006() {
```

Method `jalankanDFS_1006()` digunakan untuk melakukan pencarian jalur menggunakan algoritma Depth First Search. DFS menelusuri graph secara mendalam pada satu cabang terlebih dahulu sebelum kembali ke cabang sebelumnya. Node yang telah dikunjungi akan dicatat agar tidak terjadi pengulangan kunjungan. Jika node tujuan ditemukan, proses pencarian akan dihentikan dan jalur akan dibangun kembali. Hasil pencarian kemudian ditampilkan pada panel hasil.

#### M. Proses Rekursif DFS

```
private void dfsBFS_1006(String node, String goal, Set<String> visited, Map<String, String> parent, boolean[] found) {
```

Method ini merupakan inti dari algoritma DFS yang berjalan secara rekursif. Setiap node yang dikunjungi akan ditandai sebagai visited lalu program akan melanjutkan pencarian ke seluruh tetangga node tersebut. Jika node tujuan ditemukan maka

proses pencarian akan dihentikan. Method ini memungkinkan graph ditelusuri secara mendalam hingga mencapai node tujuan. Pendekatan rekursif membuat implementasi DFS menjadi lebih sederhana.

#### N. Rekonstruksi Jalur

```
private void rekonstruksiJalur_1006(Map<String, String> parent_1006) {
```

Method ini digunakan untuk membangun kembali jalur dari node tujuan menuju node awal berdasarkan data parent yang telah disimpan sebelumnya. Jalur disusun secara berurutan sehingga dapat ditampilkan kepada pengguna. Semua node yang termasuk dalam jalur solusi akan disimpan ke dalam variabel khusus. Data ini juga digunakan untuk memberi warna berbeda pada node dan edge yang termasuk jalur hasil pencarian. Dengan demikian jalur dapat terlihat jelas pada visualisasi graph.

#### O. Menampilkan Hasil Pencarian

```
private void displayPath_1006(String metode, boolean ditemukan) {
```

Method ini bertugas menampilkan hasil BFS atau DFS pada label yang tersedia. Informasi yang ditampilkan meliputi metode yang digunakan, jalur yang ditemukan, node yang dikunjungi, serta jumlah node yang diperiksa. Jika jalur tidak ditemukan maka program akan memberikan informasi tersebut kepada pengguna. Tampilan hasil diperbarui setiap kali proses pencarian selesai dilakukan. Dengan demikian pengguna dapat langsung melihat hasil pencarian.

#### P. Reset Data Pencarian

```
private void resetGraph_1006() {
```

Method reset digunakan untuk menghapus seluruh hasil pencarian yang sebelumnya ditampilkan pada aplikasi. Semua daftar node yang dikunjungi, jalur yang ditemukan, dan node yang berada pada jalur solusi akan dikosongkan kembali. Label hasil pencarian juga dikembalikan ke kondisi awal. Setelah proses reset

selesai, panel graph akan diperbarui. Fitur ini memungkinkan pengguna melakukan pencarian baru tanpa menutup aplikasi.

#### Q. Menampilkan Pesan Peringatan

```
private void tampilkanPesan_1006(String pesan) {
```

Method ini digunakan untuk menampilkan pesan peringatan kepada pengguna dalam bentuk dialog box. Pesan akan muncul ketika terjadi kondisi tertentu, misalnya lokasi awal dan tujuan yang dipilih sama. Dialog ditampilkan menggunakan JOptionPane. Dengan adanya peringatan ini pengguna dapat mengetahui kesalahan input yang dilakukan. Method ini membantu meningkatkan interaksi antara program dan pengguna.

#### R. Membuat Class PanelGraph\_1006

```
class PanelGraph_1006 extends JPanel {
```

Class PanelGraph\_1006 merupakan inner class yang digunakan untuk menampilkan visualisasi graph pada panel GUI. Class ini mewarisi JPanel sehingga dapat menggambar node dan edge menggunakan objek Graphics2D. Warna node dan edge akan berubah sesuai proses BFS atau DFS yang sedang dijalankan. Jalur yang ditemukan akan ditampilkan dengan warna berbeda agar mudah dikenali. Class ini berperan penting dalam menampilkan representasi visual graph kampus.

#### S. Menggambar Komponen Graph

```
protected void paintComponent(Graphics g) {
```

Method paintComponent() digunakan untuk menggambar seluruh graph pada panel visualisasi. Proses yang dilakukan meliputi penggambaran edge terlebih dahulu kemudian dilanjutkan dengan penggambaran node. Penggunaan Graphics2D membuat tampilan graph menjadi lebih halus karena mendukung antialiasing.

Setiap kali panel diperbarui, method ini akan dijalankan kembali. Dengan demikian visualisasi graph selalu sesuai dengan kondisi terbaru.

#### T. Menggambar Edge

```
private void drawEdges_1006(Graphics2D g2) {
```

Method ini digunakan untuk menggambar garis penghubung antar node pada graph. Edge yang termasuk dalam jalur hasil pencarian akan diberi warna dan ketebalan berbeda agar mudah dikenali. Program juga memastikan bahwa setiap edge hanya digambar satu kali meskipun graph bersifat tidak berarah. Penggambaran edge dilakukan berdasarkan posisi node yang telah ditentukan sebelumnya. Hasilnya adalah visualisasi jalur antar lokasi kampus.

#### U. Menggambar Node

```
private void drawNodes_1006(Graphics2D g2) {
```

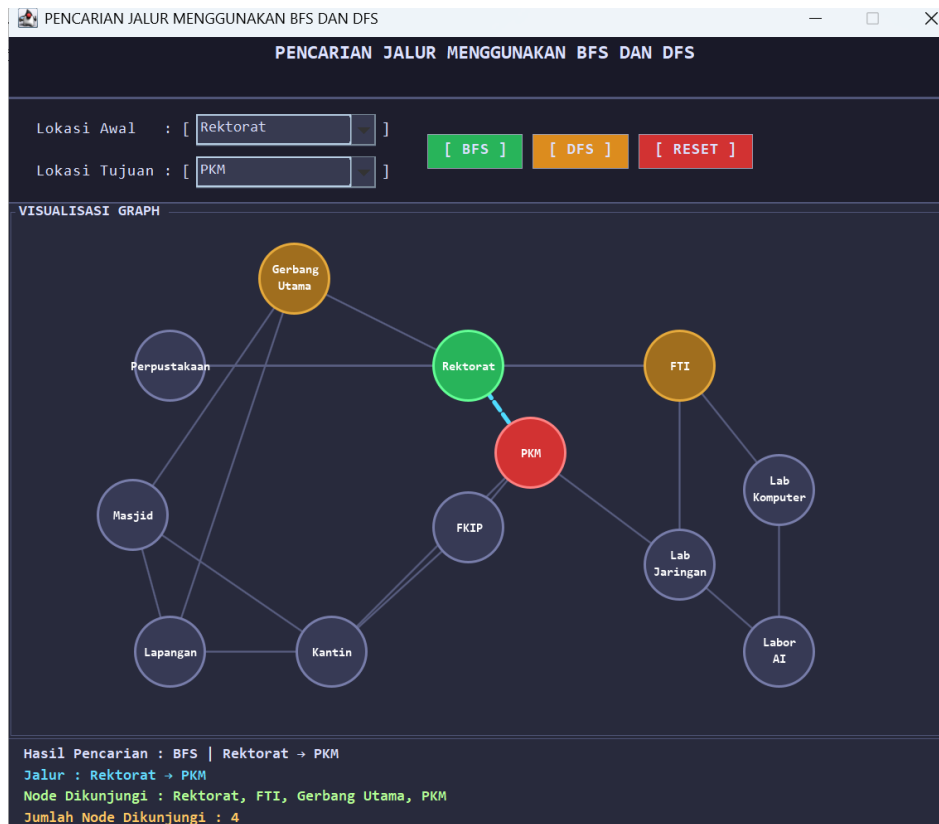
Method ini digunakan untuk menggambar seluruh node yang ada pada graph. Setiap node ditampilkan dalam bentuk lingkaran dengan warna yang berbeda sesuai statusnya. Node awal, node tujuan, node yang dikunjungi, dan node yang termasuk jalur solusi memiliki warna yang berbeda-beda. Selain menggambar lingkaran, method ini juga menampilkan nama lokasi pada setiap node. Dengan demikian pengguna dapat melihat seluruh lokasi kampus secara visual.

#### V. Method Main

```
public static void main(String[] args) {
```

Method main() merupakan titik awal eksekusi program. Pada method ini dilakukan pengaturan tampilan Swing menggunakan Look and Feel bawaan Java. Setelah itu program membuat objek PetaCampus\_1006 sehingga seluruh GUI dan data graph langsung ditampilkan. Semua proses dalam aplikasi dimulai dari method ini. Ketika program dijalankan, method main() akan dieksekusi terlebih dahulu oleh JVM.

#### W. Hasil Output Program



Program dijalankan untuk menampilkan simulasi pencarian jalur pada peta kampus menggunakan algoritma BFS (*Breadth First Search*) dan DFS (*Depth First Search*). Tampilan GUI memperlihatkan lokasi awal dan lokasi tujuan yang dipilih melalui ComboBox. Pada contoh ini, lokasi awal yang dipilih adalah **Rektorat** dan lokasi tujuan adalah **PKM**. Setelah tombol BFS ditekan, program melakukan proses penelusuran graf untuk menemukan jalur menuju tujuan. Jalur yang ditemukan kemudian ditampilkan pada panel hasil serta divisualisasikan pada graf dengan warna yang berbeda. Program juga menampilkan daftar node yang dikunjungi selama proses pencarian dan jumlah total node yang telah dikunjungi.

### Hasil yang diperoleh:

- Lokasi Awal : Rektorat
- Lokasi Tujuan : PKM
- Metode : BFS
- Jalur ditemukan : Rektorat → PKM
- Node dikunjungi : Rektorat, FTI, Gerbang Utama, PKM

- Jumlah node dikunjungi : 4

**Keterangan:**

- Node berwarna **hijau** menunjukkan lokasi awal (*start node*).
- Node berwarna **merah** menunjukkan lokasi tujuan (*goal node*).
- Garis berwarna **biru putus-putus** menunjukkan jalur yang berhasil ditemukan.
- Node berwarna **coklat** merupakan node yang telah dikunjungi selama proses pencarian.
- Informasi hasil pencarian ditampilkan pada bagian bawah aplikasi sehingga pengguna dapat melihat jalur dan jumlah node yang dikunjungi secara langsung.



## **BAB III**

### **PENUTUP**

#### **3.1 Kesimpulan**

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma Breadth First Search (BFS) dan Depth First Search (DFS) dapat diterapkan dengan baik pada struktur data graph untuk melakukan pencarian jalur antar lokasi pada peta kampus. Program yang dibuat mampu menampilkan visualisasi graph secara interaktif menggunakan Java Swing sehingga pengguna dapat melihat hubungan antar node secara jelas. Selain itu, penggunaan BFS dan DFS memungkinkan proses pencarian jalur dilakukan dengan metode yang berbeda sesuai kebutuhan. BFS menelusuri graph secara melebar sehingga efektif untuk mencari jalur terpendek pada graph tidak berbobot, sedangkan DFS menelusuri graph secara mendalam hingga mencapai node tujuan. Hasil pencarian dapat ditampilkan dalam bentuk jalur yang ditemukan, daftar node yang dikunjungi, serta jumlah node yang telah ditelusuri. Dengan demikian, praktikum ini memberikan pemahaman yang lebih mendalam mengenai implementasi graph, algoritma BFS dan DFS, serta pengembangan aplikasi GUI menggunakan bahasa pemrograman Java.

#### **3.2 Saran**

Adapun saran yang dapat diberikan untuk pengembangan program selanjutnya adalah sebagai berikut:

1. Menambahkan fitur pencarian jalur terpendek menggunakan algoritma Dijkstra.
2. Menambahkan animasi proses BFS dan DFS agar lebih mudah dipahami.
3. Menambahkan bobot pada setiap edge sehingga graph menjadi lebih realistis.
4. Menambahkan fitur penyimpanan data graph ke dalam file.
5. Mengembangkan tampilan GUI agar lebih menarik dan responsif.

## **DAFTAR PUSTAKA**

Java: The Complete Reference. New York: McGraw-Hill Education, 2018.

Data Structures and Algorithm Analysis in Java. Pearson Education, 2014